

VPP：虚拟流水线并行与 Interleaved 1F1B

原始问题：详细讲解 vpp，并且我现在的记事本网址为：<https://psgmessi.github.io/ai-infra-learning/> 以后的更新都直接在这个网址上更新，把这个方式记录进skill里，以后每次提问新的知识点都将更新到这里，包括现在这个。随后补充要求：按照 AI Infra Tutor Skill 给出完整讲解并同步更新知识图谱和线上笔记本。

提问：2026/07/27 03:08 更新：2026/07/29 19:38 重要度：5/5

一句话结论

VPP（Virtual Pipeline Parallelism，虚拟流水线并行）不是增加 GPU 数，而是把每个物理 pipeline stage 上的连续层再切成 v 个更小的 model chunks，让同一张 GPU 在调度上扮演多个逻辑 stage，并用 interleaved 1F1B 把这些 chunk 与不同 microbatch 的前后向交错执行。理想情况下，pipeline bubble 相对普通 1F1B 缩小约 v 倍；代价是 P2P 通信次数/总量约增加 v 倍、调度更复杂，并且收益依赖 stage 均衡和通信能被隐藏。

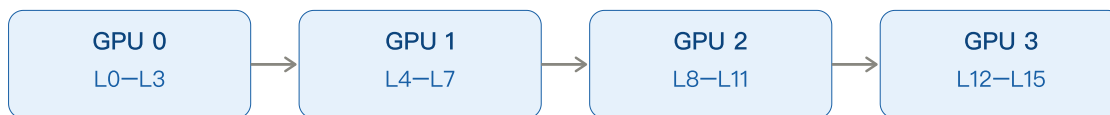
系统位置：VPP 解决什么

普通 Pipeline Parallelism（PP）把模型按层切成 p 个物理 stage，每张 GPU 只负责一个连续层块。即使使用内存友好的 1F1B，流水线启动时仍要 warm-up、结束时仍要 cool-down，因此存在 bubble。VPP 不改变模型数学结果、不改变物理 GPU 数、不替代 TP/DP/FSDP；它是在 PP 内部进一步细化 stage 并优化调度，主要解决流水线粒度太粗导致的 bubble。

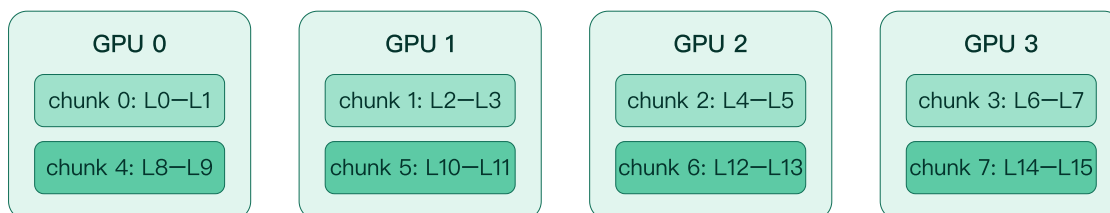
关键图：普通 PP 与 VPP 的层映射

16 层模型、4 个物理 stage：普通 PP vs VPP(v=2)

普通 PP：每张 GPU 只持有一个连续层块



VPP：每张 GPU 持有 v=2 个不连续 model chunks



逻辑前向顺序仍然不变

GPU0/chunk0 → GPU1/chunk1 → GPU2/chunk2 → GPU3/chunk3 → GPU0/chunk4 → ... → GPU3/chunk7
“virtual”指一张物理 GPU 扮演多个逻辑 stage；GPU 数没变，参数总量也没变。

模型如何切分

统一例子：L=16 个 Transformer layers，p=4 个物理 pipeline stages，v=2 个 model chunks/GPU。普通 PP 每卡 4 层：GPU0=L0-L3，GPU1=L4-L7，GPU2=L8-L11，GPU3=L12-L15。VPP 把每卡 4 层再分成两个 2 层 chunk：GPU0 持 L0-L1 和 L8-L9；GPU1 持 L2-L3 和 L10-L11；GPU2 持 L4-L5 和 L12-L13；GPU3 持 L6-L7 和 L14-L15。逻辑网络顺序没有改变，只是物理 GPU 0 在逻辑上同时扮演 virtual stage 0 和 4。

深度补充：普通 PP、1F1B 与 VPP Bubble 完整推导

一句话结论

你截图中的公式是正确的，但要真正理解，必须把三个层次分开：

1. 普通 PP 的结构：模型按层切成 p 个连续物理 Stage，一个 Microbatch 的 Forward 依次穿过 Stage 0→1→...→p-1，Backward 再反向返回。

2. **1F1B 的作用**：它把不同 Microbatch 的 Forward/Backward 交错排列，主要减少需要长期保存的 Activation；在经典同步 Flush 语义下，它与 GPipe 的理论 Bubble 大小相同。
3. **VPP 的变化**：每张物理 GPU 不再只持有一个大 Stage，而是持有 v 个更小的 Model Chunks。每个 Chunk 的时间约缩短到原来的 $1/v$ ，所以填充/排空气泡的持续时间约缩小 v 倍；但每张 GPU 的总有效计算量并没有减少。

最容易记错的地方是：

VPP 把 Bubble 缩小约 v 倍
≠ 把整个 Step 时间缩小 v 倍

在统一例子 $p=4, m=8, v=2, t_f=t_b=1$ 中：

普通 1F1B: Useful=16, Bubble=6, Total=22
VPP $v=2$: Useful=16, Bubble=3, Total=19

理想加速 = $22 \div 19 \approx 1.158\times$

Bubble 虽然减半，但总时间只理想下降约 13.6%，吞吐理想提升约 15.8%。

一、统一符号、假设和计数口径

整节使用同一组参数：

$L = 16$ Transformer 层数
 $p = 4$ 物理 Pipeline Stage / GPU 数
 $m = 8$ 一个训练 Step 中的 Microbatch 数
 $v = 2$ 每个物理 GPU 的 Model Chunk 数

 $t_f = 1 \text{ time slot}$ 普通物理 Stage 处理一个 Microbatch Forward 的时间
 $t_b = 1 \text{ time slot}$ 普通物理 Stage 处理一个 Microbatch Backward 的时间

核心理想化假设：

- 四个物理 Stage 计算完全均衡；
- 暂时忽略 P2P 通信时间；
- Forward/Backward 各使用一个单位时隙；
- 所有 Microbatch 使用同一版本权重；
- 一个 Step 的所有 Microbatch 完成后才执行 Optimizer Step；
- 讨论的是同步 Flush 型流水线，而不是允许权重陈旧的异步 PipeDream。

这些公式首先描述**调度上限**。真实时间还要加入：

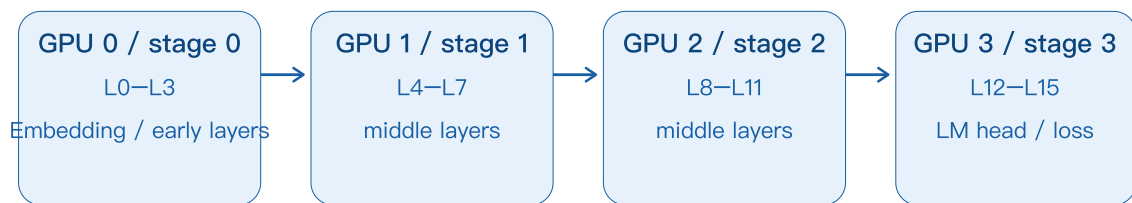
暴露的 P2P 通信
+ Stage 不均衡
+ Kernel Launch / 调度开销
+ Stream 同步
+ 通信与计算资源争抢

二、普通 Pipeline Parallelism 的结构

16 层 Transformer 使用 $p=4$ 做普通 PP:

GPU 0 / Stage 0: L0-L3
GPU 1 / Stage 1: L4-L7
GPU 2 / Stage 2: L8-L11
GPU 3 / Stage 3: L12-L15

普通 PP: 模型沿深度切成 4 个连续 stage



Forward: 发送 activation [microbatch, sequence, hidden]



Backward: 发送 activation gradient, 方向相反



Microbatch 流水化

MB0 离开 stage 0 后, stage 0 可立刻处理 MB1; 不同 stage 同时计算不同 microbatch。

一个 Microbatch 的 Forward:

```
Input
↓
Stage 0: L0-L3
  ↓ 发送 Activation
Stage 1: L4-L7
  ↓ 发送 Activation
Stage 2: L8-L11
  ↓ 发送 Activation
Stage 3: L12-L15 + Loss
```

Backward 方向相反:

```
Stage 3 计算输入梯度
  ↓ 发送 Activation Gradient
Stage 2
  ↓
Stage 1
  ↓
Stage 0
```

Stage 间通常不发送模型参数，而是发送边界 Activation 或对应的 Activation Gradient。典型张量 Shape：

```
[micro_batch_size, sequence_length, hidden_size]
```

普通 PP 的关键是把大 Batch 切成  个 Microbatches。MB0 离开 Stage 0 后，Stage 0 不必等待它跑完整个模型，可以立刻处理 MB1；于是不同 GPU 同时处理不同 Microbatch：

```
某一时刻：
Stage 0 在算 MB3
Stage 1 在算 MB2
Stage 2 在算 MB1
Stage 3 在算 MB0
```

这才形成流水线并行。

三、为什么会有 Pipeline Bubble

流水线开始时，后面的 Stage 没有输入：

```
time 0: 只有 Stage 0 能算 MB0
Time 1: Stage 0 算 MB1, Stage 1 才拿到 MB0
Time 2: Stage 2 才第一次拿到 MB0
Time 3: Stage 3 才第一次拿到 MB0
```

因此，对 ：

```
Stage 3 在第一次开始 Forward 前等待了 p-1 = 3 个 Forward 时隙
```

这叫 **Pipeline Fill / Warm-up Bubble**。

结束时情况反过来。最后一个 Stage 可以先完成 Backward，但梯度还要逐级返回：

```
Stage 3 完成最后一个 Backward
→ Stage 2 才能继续对应 Backward
→ Stage 1
→ Stage 0 最后完成
```

这叫 **Pipeline Drain / Cool-down Bubble**。

所以普通同步 PP 的 Bubble 来自依赖链两端：

```
开始时：需要  $p-1$  个额外 Forward Stage 间隔  
结束时：需要  $p-1$  个额外 Backward Stage 间隔
```

四、GPipe: All-Forward-All-Backward

最直观的调度是 GPipe 风格：

```
先完成所有 Microbatch 的 Forward  
F0, F1, F2, ..., F7  
  
再完成所有 Microbatch 的 Backward  
B7, B6, B5, ..., B0
```

在平衡假设下：

```
Forward Makespan =  $(m + p - 1) \times t_f$   
Backward Makespan =  $(m + p - 1) \times t_b$ 
```

总时间：

```
T_total_gpipe  
=  $(m+p-1)t_f + (m+p-1)t_b$   
=  $(m+p-1)(t_f+t_b)$ 
```

每个 Stage 真正必须做的有效计算只有：

```
m 次 Forward + m 次 Backward
```

因此：

```
T_useful =  $m(t_f+t_b)$ 
```

两者相减就是 Bubble：

```
T_bubble  
=  $T_{total} - T_{useful}$   
=  $(m+p-1)(t_f+t_b) - m(t_f+t_b)$   
=  $(p-1)(t_f+t_b)$ 
```

GPipe 的主要问题不是 Bubble 比 1F1B 更大，而是 Activation 生命周期很长：在开始 Backward 前，已经做完所有 m 个 Forward，所以早期 Stage 需要保留接近 m 份尚未反向的 Activation。

五、1F1B 到底如何运行

1F1B = One Forward, One Backward。

它不是从第一个时隙开始所有 Stage 都严格 **F,B,F,B**。完整调度分三段：

1. **Warm-up**：先做若干 Forward，把流水线填起来；
2. **Steady State**：交替执行一个 Forward 和一个更早 Microbatch 的 Backward；
3. **Cool-down**：没有新 Forward 后，完成剩余 Backward。

对物理 Stage **s**（从 0 开始），经典非交错 1F1B 的 Warm-up Forward 数量近似为：

$$w_s = p - s - 1$$

在 **p=4** 时：

Stage	Warm-up Forward 数	为什么
Stage 0	3	要先把足够多的 Microbatch 推向后面，才能等到第一个 Backward 返回
Stage 1	2	比 Stage 0 少一跳
Stage 2	1	离最后 Stage 只差一跳
Stage 3	0	它算出 Loss 后就能立刻开始该 Microbatch 的 Backward

p=4、m=8 时，各 Stage 的逻辑任务顺序是：

```
Stage 0:
F0 F1 F2 | F3 B0 F4 B1 F5 B2 F6 B3 F7 B4 | B5 B6 B7

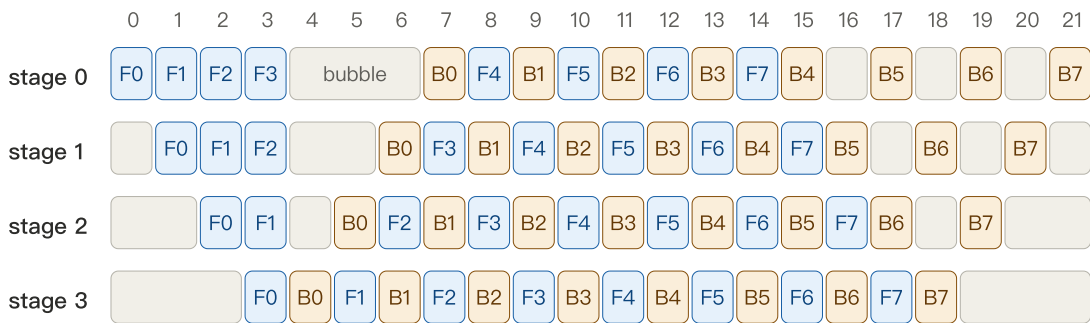
Stage 1:
F0 F1   | F2 B0 F3 B1 F4 B2 F5 B3 F6 B4 F7 B5 | B6 B7

Stage 2:
F0      | F1 B0 F2 B1 F3 B2 F4 B3 F5 B4 F6 B5 F7 B6 | B7

Stage 3:
F0 B0 F1 B1 F2 B2 F3 B3 F4 B4 F5 B5 F6 B6 F7 B7
```

竖线只是标识 Warm-up、Steady State、Cool-down 的逻辑分段；真实时间轴中还要满足前后 Stage 的依赖，因此会出现等待空槽。

普通 1F1B: $p=4, m=8, t_f=t_b=1$ slot



蓝色 F: 前向 橙色 B: 反向 灰色: bubble / idle

每个 stage: 16 个有效槽 + 6 个空闲槽; 总长度 22。利用率 = $16/22 = 72.73\%$ 。

为什么 Stage 0 做完 F3 后不能立刻 B0?

Stage 0 的 B0 依赖完整的反向链:

```
MB0 必须先完成 Stage 3 的 F0 和 Loss
→ Stage 3 完成 B0
→ Stage 2 完成 B0
→ Stage 1 完成 B0
→ Stage 0 才能执行 B0
```

因此, 前向波刚到达最后 Stage 后, 反向波还要逐级返回。图中 Stage 0 在 F3 后出现 3 个灰色空槽, 正是等待 B0 梯度返回。

1F1B 的核心收益是什么?

某个 Microbatch 完成当前 Stage 的 Backward 后, 对应 Forward Activation 就可以释放。

因此 Outstanding Forward Activations 从 GPipe 的约 m 份下降到不超过约 p 份:

```
GPipe Activation Memory  $\approx O(m)$ 
1F1B Activation Memory  $\approx O(p)$ 
```

当 $m \gg p$ 时, 显存差异非常大。

六、为什么普通 1F1B 的 Bubble 仍是 $(p-1)(t_f+t_b)$

1F1B 把 Backward 提前了, 但没有消除两条物理依赖:

```
Forward 仍要从 Stage 0 逐级传播到 Stage p-1
```


Backward 仍要从 Stage p-1 逐级返回 Stage 0

所以：

Forward 端额外填充时间 = $(p-1)t_f$
Backward 端额外排空时间 = $(p-1)t_b$

相加：

$T_{bubble} = (p-1)t_f + (p-1)t_b$
 $= (p-1)(t_f+t_b)$

这也是为什么普通 GPipe 与普通 1F1B 在理想平衡模型下具有相同 Bubble：

- GPipe：先全 F 再全 B；
- 1F1B：把 F/B 在中间阶段交错；
- 但首端 Fill 和尾端 Drain 的依赖长度没有变化。

1F1B 主要优化的是 Activation 生命周期，而不是普通非交错 Pipeline 的理论 Bubble。

七、为什么 $T_{useful} = m(t_f+t_b)$

我们从一张物理 GPU / 一个物理 Stage 的视角计数。

一个 Step 有 m 个 Microbatch。每个 Stage 必须对每个 Microbatch 做：

1 次 Forward，耗时 t_f
1 次 Backward，耗时 t_b

因此，一个 Stage 的有效工作：

T_{useful}
 $= m \times t_f + m \times t_b$
 $= m(t_f+t_b)$

这里没有乘 p ，因为我们计算的是单个 Stage 的时间线长度。所有 Stage 并行运行，它们在平衡假设下都有相同的有效工作量。

如果改用整个集群的 GPU-Time 口径，则：

Cluster Useful GPU-Time = $p \times m(t_f+t_b)$

实际总 GPU-Time：

$$\text{Cluster Total GPU-Time} = p \times T_{\text{total}}$$

分子分母都有 p ，因此 Bubble 比例不变。

八、普通 PP 的总时间和两种 Bubble 比例

普通 GPipe / 普通 1F1B:

$$\begin{aligned} T_{\text{useful}} &= m(t_f + t_b) \\ T_{\text{bubble}} &= (p-1)(t_f + t_b) \\ \\ T_{\text{total}} &= T_{\text{useful}} + T_{\text{bubble}} \\ &= (m+p-1)(t_f + t_b) \end{aligned}$$

口径 1: Bubble 相对理想有效计算

$$\begin{aligned} \text{Bubble / Useful} &= T_{\text{bubble}} \div T_{\text{useful}} \\ &= (p-1)(t_f + t_b) \div [m(t_f + t_b)] \\ &= (p-1) \div m \end{aligned}$$

这个比值可以大于 100%，因为它表示“额外浪费相当于理想计算的多少”，不是总时间占比。

口径 2: Bubble 占实际总时间

$$\begin{aligned} \text{Bubble / Actual Total} &= T_{\text{bubble}} \div (T_{\text{useful}} + T_{\text{bubble}}) \\ &= (p-1) \div (m+p-1) \end{aligned}$$

它一定在 0% 到 100% 之间。

Pipeline 利用率

$$\begin{aligned} \text{Utilization} &= T_{\text{useful}} \div T_{\text{total}} \\ &= m \div (m+p-1) \end{aligned}$$

并且：

$$\text{Utilization} + \text{Bubble/Actual Total} = 1$$

九、普通 1F1B 数值算例：p=4、m=8

代入：

```
p = 4  
m = 8  
t_f = 1 slot  
t_b = 1 slot
```

有效计算：

```
T_useful  
= 8 × (1+1)  
= 16 slots
```

Bubble：

```
T_bubble  
= (4-1) × (1+1)  
= 3 × 2  
= 6 slots
```

实际总时间：

```
T_total = 16 + 6 = 22 slots
```

两种比例：

```
Bubble / Useful  
= 6 ÷ 16  
= 37.5%
```

```
Bubble / Actual Total  
= 6 ÷ 22  
≈ 27.27%
```

利用率：

```
Utilization  
= 16 ÷ 22  
≈ 72.73%
```

这正对应时隙图：每个 Stage 在共享的 22-slot 时间轴中，都做了 16 个有效任务，并累计空闲 6 个 Slot。不同 Stage 的空闲位置不同，但总空闲时间相同。

十、VPP 在普通 PP 基础上改变了什么

普通 PP：每张 GPU 持有一个连续大 Stage。

```
GPU 0: L0-L3
GPU 1: L4-L7
GPU 2: L8-L11
GPU 3: L12-L15
```

VPP **v=2**：把每张 GPU 的 4 层拆成两个 2 层 Chunk，并按全局层顺序交错映射：

```
GPU 0: chunk 0 = L0-L1,   chunk 4 = L8-L9
GPU 1: chunk 1 = L2-L3,   chunk 5 = L10-L11
GPU 2: chunk 2 = L4-L5,   chunk 6 = L12-L13
GPU 3: chunk 3 = L6-L7,   chunk 7 = L14-L15
```

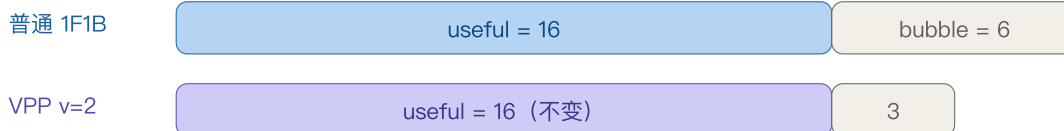
VPP(v=2)：每张物理 GPU 扮演两个虚拟 stage



一个 microbatch 的逻辑前向路径



相同有效工作，不同 bubble



每个 chunk 时间： t_f/v 、 t_b/v ；物理 GPU 仍做 v 个 chunks，总有效计算不变。

Bubble：6 $\rightarrow$ 3；总时间：22 $\rightarrow$ 19；理想加速：22/19 $\approx$ 1.158，而不是 2 $\times$ 。

代价：逻辑 stage 边界增多，P2P 通信量约增 v 倍；真实收益低于理想值。

一个 Microbatch 的逻辑 Forward 路径变成：

```
C0/GPU0 -> C1/GPU1 -> C2/GPU2 -> C3/GPU3
          -> C4/GPU0 -> C5/GPU1 -> C6/GPU2 -> C7/GPU3
```

同一张 GPU 在不同时间处理不同 Chunk，所以它在调度上扮演多个虚拟 Stage。

十一、为什么 VPP 的有效计算不变

普通 PP 中，一个物理 Stage 对一个 Microbatch 的总计算时间：

$$t_f + t_b$$

拆成 v 个大小相近的 Chunk 后，每个 Chunk 的时间约为：

$$\begin{aligned} t_{f_chunk} &\approx t_f \div v \\ t_{b_chunk} &\approx t_b \div v \end{aligned}$$

但是一张物理 GPU 需要执行自己持有的全部 v 个 Chunks。

因此，对一个 Microbatch：

$$\begin{aligned} \text{每张 GPU 的有效计算} \\ &= v \times (t_f/v + t_b/v) \\ &= t_f + t_b \end{aligned}$$

对 m 个 Microbatch：

$$\begin{aligned} T_{\text{useful_vpp}} \\ &= m \times v \times (t_f/v + t_b/v) \\ &= m(t_f + t_b) \end{aligned}$$

所以 VPP 没有减少模型 FLOP，也没有减少每张 GPU 负责的层数。它改变的是任务粒度与时间排列。

十二、为什么 VPP Bubble 缩小约 v 倍

普通 PP 的 Fill/Drain 空档以“完整物理 Stage 计算时间”为粒度：

$$\begin{aligned} \text{Forward 空档粒度} &= t_f \\ \text{Backward 空档粒度} &= t_b \end{aligned}$$

VPP 把大 Stage 切成 v 份后，Interleaved 1F1B 可以用其他 Chunk/Microbatch 的 Ready Task 填充原来的大空档。剩余不可消除的边缘空档以 Chunk 时间为粒度：

$$\begin{aligned} \text{Forward Chunk 时间} &\approx t_f/v \\ \text{Backward Chunk 时间} &\approx t_b/v \end{aligned}$$

仍有 $p-1$ 个物理 Rank 间隔需要填充和排空，因此理想 Bubble：

```
T_bubble_vpp
≈ (p-1)(t_f/v + t_b/v)
= (p-1)(t_f+t_b)/v
```

为什么不是 $(pv-1)(t_f+t_b)/v$?

pv 是逻辑虚拟 Stage 数，它影响单个 Microbatch 穿过完整模型的延迟和通信边界数。但这里计算的是多个 Microbatch 交错执行时，每张物理 GPU 的批次吞吐时间线中的未填充空档。

额外的虚拟 Stage 路径并不是全部串行暴露：它们会与其他 Microbatch、其他 Chunk 的计算交错。经典 Megatron Interleaved 调度的效果，就是把不可避免的 Pipeline Edge 空档从完整 Stage 时间压缩到 Chunk 时间。

因此要区分：

```
单 Microbatch 端到端延迟
≠ 一个 Batch 的 Pipeline Bubble
≠ 每张物理 GPU 的有效计算量
```

十三、VPP 两种 Bubble 比例

VPP 有效时间仍然是：

```
T_useful_vpp = m(t_f+t_b)
```

Bubble：

```
T_bubble_vpp = (p-1)(t_f+t_b)/v
```

实际总时间：

```
T_total_vpp
= m(t_f+t_b) + (p-1)(t_f+t_b)/v
= [m + (p-1)/v](t_f+t_b)
```

Bubble 相对 Useful

```
Bubble / Useful
= [(p-1)(t_f+t_b)/v] ÷ [m(t_f+t_b)]
= (p-1) ÷ (mv)
```

Bubble 占实际总时间

$$\begin{aligned} \text{Bubble / Actual Total} &= [(p-1)/v] \div [m+(p-1)/v] \\ &= (p-1) \div (mv+p-1) \end{aligned}$$

利用率

$$\begin{aligned} \text{Utilization_vpp} &= mv \div (mv+p-1) \end{aligned}$$

十四、VPP 数值算例：p=4、m=8、v=2

Chunk 时间：

$$\begin{aligned} t_f_chunk &= 1 \div 2 = 0.5 \text{ slot} \\ t_b_chunk &= 1 \div 2 = 0.5 \text{ slot} \end{aligned}$$

有效计算不变：

$$\begin{aligned} T_useful_vpp &= 8 \times 2 \times (0.5+0.5) \\ &= 16 \text{ slots} \end{aligned}$$

Bubble：

$$\begin{aligned} T_bubble_vpp &= (4-1) \times (1+1) \div 2 \\ &= 3 \text{ slots} \end{aligned}$$

总时间：

$$T_total_vpp = 16 + 3 = 19 \text{ slots}$$

比例：

$$\text{Bubble / Useful} = 3 \div 16 = 18.75\%$$

$$\text{Bubble / Actual Total} = 3 \div 19 \approx 15.79\%$$

$$\text{Utilization} = 16 \div 19 \approx 84.21\%$$

对比：

指标	普通 1F1B	VPP v=2	变化
Useful	16	16	不变
Bubble	6	3	减半
Total	22	19	减少 3
Bubble/Actual	27.27%	15.79%	下降 11.48 个百分点
利用率	72.73%	84.21%	上升 11.48 个百分点
理想吞吐提升	1×	22/19≈1.158×	约 15.8%

这说明：Bubble 减半不代表 Step Time 减半。因为 Useful 16 完全没有变，VPP 只减少了原来 6 个空闲 Slot 中的 3 个。

即使令 $V \rightarrow \infty$ 、Bubble 理想趋近 0，本例的绝对加速上限也只有：

```
Speedup_max  
= 22 ÷ 16  
= 1.375×
```

这还是忽略通信与调度开销的上限。

十五、VPP 为什么不是免费午餐

VPP 增加了逻辑 Stage 边界。普通 PP 的 Forward 路径：

```
GPU0 → GPU1 → GPU2 → GPU3
```

VPP $v=2$ ：

```
GPU0 → GPU1 → GPU2 → GPU3  
→ GPU0 → GPU1 → GPU2 → GPU3
```

所以 P2P Send/Recv 次数和总通信量在经典模型中约增加 V 倍。

真实 Step Time 更接近：

```
T_real_vpp  
≈ T_useful  
+ T_bubble_vpp  
+ T_extra_p2p_exposed  
+ T_stage_imbalance  
+ T_schedule_overhead
```


是否启用 VPP，应判断：

```
节省的 Bubble 时间
>
新增的暴露通信时间 + 调度开销
```

本例理论节省 3 Slots。如果新增 P2P 和调度暴露超过 3 Slots，VPP 反而更慢。

经典 Megatron Interleaved 1F1B 还要求 Microbatch 数满足调度约束；NVIDIA 文章给出的典型要求是：

```
m mod p = 0
```

不同 Megatron/NeMo 版本还可能有额外整除和模型切分限制，应以实际版本断言为准。

十六、1F1B 调度伪代码

下面是帮助理解的简化伪代码，不代表某个 Megatron 版本的逐行源码：

```
def schedule_1f1b(stage_id, p, microbatches):
    m = len(microbatches)
    warmup = min(p - stage_id - 1, m)

    # 1. Warm-up: 只做 Forward
    for i in range(warmup):
        x = recv_forward(i)
        y = forward(i, x)
        send_forward(i, y)
        stash_activation(i)

    # 2. Steady state: 一个较新的 Forward + 一个较早的 Backward
    for i in range(warmup, m):
        x = recv_forward(i)
        y = forward(i, x)
        send_forward(i, y)
        stash_activation(i)

        old = i - warmup
        dy = recv_backward(old)
        dx = backward(old, dy)
        release_activation(old)
        send_backward(old, dx)

    # 3. Cool-down: 完成剩余 Backward
    for old in range(m - warmup, m):
        dy = recv_backward(old)
        dx = backward(old, dy)
        release_activation(old)
        send_backward(old, dx)
```

关键点：

- Forward 和 Backward 通常属于不同 Microbatch；
- Last Stage 的 Warm-up 为 0，所以可以 **F0→B0→F1→B1**；

- Stage 0 的 Warm-up 最长，因为第一个梯度需要走最远的返回路径；
- Optimizer Step 在整个 Microbatch Group Flush 后执行，避免不同 Microbatch 使用不同版本权重。

深度补充：1F1B 伪代码逐行解释与真实 Forward 调用

一句话结论

截图中的代码是为了表达调度思想而写的**教学伪代码**，不是 Megatron Core 可以直接运行的源码。它把生产实现中的四类对象压缩成了几个虚构函数：

```
通信: recv_forward / send_forward / recv_backward / send_backward
计算: forward / backward
状态: stash_activation / release_activation
调度: warm-up / steady state / cool-down 循环
```

在当前 Megatron Core 中，真实对应关系更接近：

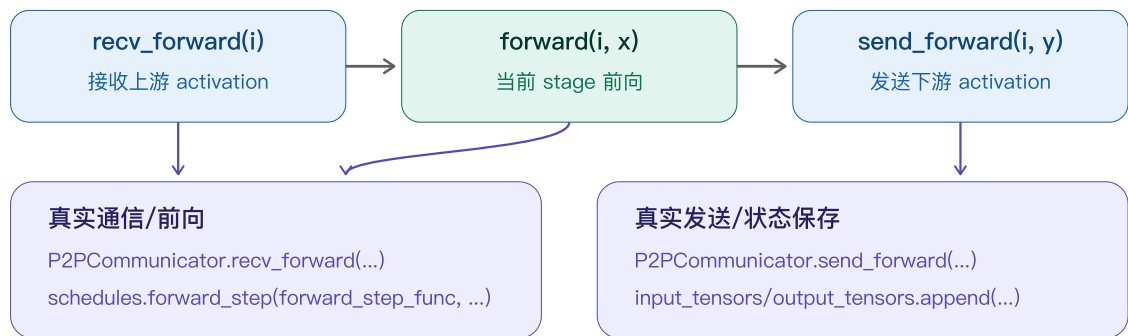
教学伪代码	Megatron Core 真实对应
<code>recv_forward(i)</code>	<code>P2PCommunicator.recv_forward(tensor_shapes, is_first_stage)</code>
<code>forward(i, x)</code>	<code>schedules.forward_step(forward_step_func, data_iterator, model, ..., input_tensor=x)</code>
<code>send_forward(i, y)</code>	<code>P2PCommunicator.send_forward(output_tensors, is_last_stage)</code>
<code>stash_activation(i)</code>	<code>input_tensors.append(input_tensor) + output_tensors.append(output_tensor)</code> ；Autograd Graph 由 PyTorch 保存在 Tensor/ <code>grad_fn</code> 中
<code>recv_backward(old)</code>	<code>P2PCommunicator.recv_backward(...)</code> ；Steady State 中通常融合进 <code>send_forward_recv_backward(...)</code>
<code>backward(old, dy)</code>	<code>schedules.backward_step(input_tensor, output_tensor, output_tensor_grad, config)</code>
<code>release_activation(old)</code>	从 FIFO <code>pop(0)</code> 后执行 Backward，引用释放时计算图可回收；Megatron 还可能更早调用 <code>deallocate_output_tensor()</code> 伪释放已发送的输出数据
<code>send_backward(old, dx)</code>	<code>P2PCommunicator.send_backward(input_tensor_grads, is_first_stage)</code> ；非最后一轮通常融合为 <code>send_backward_recv_forward(...)</code>

最重要的区别是：

`stash_activation(i)` 并不是把 Activation 复制到某个隐藏仓库；真实实现只是在 FIFO 中保存当前 Microbatch 的 `input_tensor` 和 `output_tensor` 引用，而 PyTorch

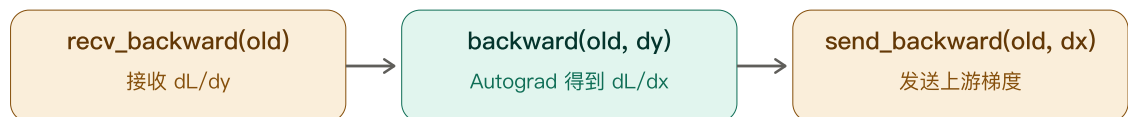
Autograd Graph 已经挂在 `output_tensor.grad_fn` 上。

教学伪代码是概念压缩；生产实现把状态、通信和计算拆成多个对象



`stash_activation(i)` 的真实含义

不是复制一个 Tensor 到神秘缓存；而是 FIFO 保存 `input_tensor + output_tensor/grad_fn`，等待旧 microbatch 反向。



Steady state 会组合通信：`send_forward_recv_backward + send_backward_recv_forward`，以减少串行等待。

一、先给伪代码加上行号

```
01 def schedule_1f1b(stage_id, p, microbatches):
02     m = len(microbatches)
03     warmup = min(p - stage_id - 1, m)
04
05     # 1. Warm-up: 只做 Forward
06     for i in range(warmup):
07         x = recv_forward(i)
08         y = forward(i, x)
09         send_forward(i, y)
10         stash_activation(i)
11
12     # 2. Steady state: 一个较新的 Forward + 一个较早的 Backward
13     for i in range(warmup, m):
14         x = recv_forward(i)
15         y = forward(i, x)
16         send_forward(i, y)
17         stash_activation(i)
18
19     old = i - warmup
20     dy = recv_backward(old)
21     dx = backward(old, dy)
22     release_activation(old)
23     send_backward(old, dx)
```

```

21     # 3. Cool-down: 完成剩余 Backward
22     for old in range(m - warmup, m):
23         dy = recv_backward(old)
24         dx = backward(old, dy)
25         release_activation(old)
26         send_backward(old, dx)

```

下面逐行解释。

二、第 1 - 3 行：计算当前 Stage 要 Warm-up 几次

第 1 行

```
def schedule_1f1b(stage_id, p, microbatches):
```

教学含义：为某一个 Pipeline Stage 执行一个训练 Step 的全部 Microbatch 调度。

参数：

参数	含义
<code>stage_id</code>	当前物理 Pipeline Stage 的编号，从 0 开始
<code>p</code>	物理 Pipeline Stage 总数
<code>microbatches</code>	当前训练 Step 被拆分后的 Microbatch 集合

真实 Megatron Core 不会只传这三个参数。非交错 1F1B 入口是：

```

forward_backward_pipelining_without_interleaving(
    forward_step_func=...,
    data_iterator=...,
    model=...,
    num_microbatches=...,
    seq_length=...,
    micro_batch_size=...,
    forward_only=False,
    p2p_communicator=...,
    ...
)

```

VPP/Interleaved 1F1B 使用：

```
forward_backward_pipelining_with_interleaving(...)
```

真实调度器还要知道：

- 当前 Stage 对应的模型或 Model Chunks；
- 数据迭代器；

- P2P Process Group;
- 通信 Tensor Shape;
- 是否 First/Last Stage;
- 是否 Forward Only;
- Activation Checkpoint 策略;
- 梯度同步策略;
- Virtual Pipeline Rank。

第 2 行

```
m = len(microbatches)
```

表示本 Step 一共有多少个 Microbatch。

在 Megatron 中通常不把一个 Python List `microbatches` 直接传给调度器，而是传：

```
num_microbatches=m
data_iterator=data_iterator
```

每次 `forward_step_func` 被调用时，First/Last Stage 根据需要从 `data_iterator` 中读取当前 Microbatch。

第 3 行

```
warmup = min(p - stage_id - 1, m)
```

含义：当前 Stage 在第一次能够执行 Backward 之前，需要先做多少个 Forward。

若：

```
p = 4
```

则：

stage_id	p-stage_id-1	Warm-up 数量
0	3	3
1	2	2
2	1	1
3	0	0

为什么 Last Stage 为 0? 因为它执行完一个 Microbatch Forward 并计算 Loss 后, 就能立即从 Loss 开始该 Microbatch 的 Backward。

为什么要和 `m` 取最小值? 如果 Microbatch 总数比所需 Warm-up 还少, 不能执行超过 `m` 次 Forward。

真实源码更接近:

```
num_warmup_microbatches = (  
    p2p_communicator.total_stages  
    - p2p_communicator.current_stage  
    - 1  
)  
num_warmup_microbatches = min(  
    num_warmup_microbatches,  
    num_microbatches,  
)
```

生产代码使用 `P2PCommunicator` 维护的 Stage 信息, 而不是把 `stage_id`、`p` 随意传进函数。

三、第 4 - 9 行: Warm-up 只做 Forward

第 5 行

```
for i in range(warmup):
```

`i` 是当前 Forward Microbatch 编号。

如果 Stage 0 的 `warmup=3` :

```
i = 0, 1, 2
```

这三个 Microbatch 的 Forward 结果会进入“尚未 Backward”的队列。

第 6 行

```
x = recv_forward(i)
```

含义: 取得当前 Stage 的 Forward 输入。

不同 Stage 的行为不同:

Stage	x 从哪里来
First Stage	不从前一个 Pipeline Rank 接收；输入来自本地 Dataset 的 Tokens/Embedding
Middle Stage	从前一个 Pipeline Rank 接收 Hidden States
Last Stage	同样从前一个 Rank 接收 Hidden States，同时本地取得 Labels/Loss Mask

真实 API：

```
input_tensor = p2p_communicator.recv_forward(
    recv_tensor_shapes,
    p2p_communicator.is_pp_first_stage,
)
```

First Stage 时，Communicator 知道没有前驱，一般返回 None 或与模型接口一致的空输入；模型会从本地 tokens 开始。

i 通常不会直接发送给远端。通信双方通过相同的调度顺序，知道当前收发的是哪个 Microbatch。若 Variable Sequence Length 开启，还可能先通信 Tensor Shape。

第 7 行

```
y = forward(i, x)
```

教学上表示：当前 Stage 对 Microbatch i 执行本地层的 Forward。

真实调用分两层：

```
Megatron 调度器的 forward_step(...)
↓
用户/训练脚本提供的 forward_step_func(data_iterator, model)
↓
model(...)
```

真实调度层调用大致是：

```
output_tensor, num_tokens = forward_step(
    forward_step_func,
    data_iterator,
    model,
    num_microbatches,
    input_tensor,
    forward_data_store,
    config,
    cp_group_size,
    current_microbatch=i,
    is_last_stage=p2p_communicator.is_pp_last_stage,
)
```

forward_step() 在调用用户函数之前，会把接收到的 Pipeline Activation 注入模型，概念上相当于：

```
model.set_input_tensor(input_tensor)
output_tensor, loss_func = forward_step_func(data_iterator, model)
```

对于 Last Stage，它还会调用 `loss_func(output_tensor)` 得到 Loss 和日志数据。

第 8 行

```
send_forward(i, y)
```

含义：把当前 Stage 的输出 Activation 发给下一个 Stage。

真实 API：

```
p2p_communicator.send_forward(
    output_tensor,
    p2p_communicator.is_pp_last_stage,
)
```

- 非 Last Stage：发送 Hidden States；
- Last Stage：没有下一 Stage，不发送；它计算 Loss。

输出典型 Shape：

```
[sequence_length, micro_batch_size, hidden_size]
```

或框架内部采用的其他 Layout。若启用 Sequence/Context Parallel，通信 Shape 可能相应变化。

第 9 行

```
stash_activation(i)
```

这是最容易误解的一行。

它不是：

```
把 y 复制一份到额外的 Activation Cache
```

真实需要保留的是执行 Backward 所必需的计算图上下文。Megatron 非交错 1F1B 维护两个 FIFO：

```
input_tensors = []
output_tensors = []

input_tensors.append(input_tensor)
output_tensors.append(output_tensor)
```


`output_tensor` 本身带有 PyTorch Autograd 的：

```
grad_fn
```

它间接引用本 Stage Forward 所需的计算图和保存张量。

Megatron 在把输出发送给下一 Stage 后，还可能调用：

```
deallocate_output_tensor(  
    output_tensor,  
    config.deallocate_pipeline_outputs,  
)
```

这个函数会把已经发送出去的 `output_tensor.data` 伪释放成很小的 Tensor，但保留 `grad_fn`，降低 Pipeline Output 占用。它不等于释放整个 Autograd Graph。

Warm-up 完成后，若 `warmup=3`：

```
input_tensors = [input_0, input_1, input_2]  
output_tensors = [output_0, output_1, output_2]
```

这些都是等待 Backward 的 Outstanding Microbatches。

四、第 10 – 20 行：Steady State 为什么同时处理两个 Microbatch

第 11 行

```
for i in range(warmup, m):
```

继续对尚未 Forward 的 Microbatch 执行循环。

例如：

```
warmup = 3  
m = 8
```

则：

```
i = 3, 4, 5, 6, 7
```

每轮做两件不同的事：

```
较新的 Microbatch i: Forward
```

较早的 Microbatch old: Backward

第 12 – 15 行

```
x = recv_forward(i)
y = forward(i, x)
send_forward(i, y)
stash_activation(i)
```

与 Warm-up 相同，对新 Microbatch 做 Forward 并放入 Outstanding FIFO。

但生产实现通常不会把发送 Forward、接收 Backward 写成两个完全串行调用，而会组合成：

```
output_tensor_grad = (
    p2p_communicator.send_forward_recv_backward(
        output_tensor,
        send_tensor_shapes,
        p2p_communicator.is_pp_last_stage,
    )
)
```

它同时做：

```
向下一 Stage 发送当前新 Microbatch 的 Output
从下一 Stage 接收某个旧 Microbatch 的 dL/dOutput
```

第 16 行

```
old = i - warmup
```

教学代码通过算术计算当前应 Backward 的旧 Microbatch。

例如：

```
warmup = 3

i=3 → old=0
i=4 → old=1
i=5 → old=2
```

生产代码通常不依赖这个整数公式保存状态，而是使用 FIFO：

```
input_tensor = input_tensors.pop(0)
output_tensor = output_tensors.pop(0)
```

队首天然就是最早尚未完成 Backward 的 Microbatch。

FIFO 更可靠，因为真实实现还要处理：

- Virtual Pipeline Model Chunks;
- Activation Checkpoint;
- 多输入/多输出 Tensor;
- Variable Sequence Length;
- Encoder-Decoder;
- Multimodule Pipeline。

第 17 行

```
dy = recv_backward(old)
```

`dy` 是：

```
dy =  $\partial \text{Loss} / \partial y$ 
```

即 Loss 对当前 Stage 输出 Activation 的梯度，由下一个 Stage 计算后发送回来。

真实 API：

```
output_tensor_grad = p2p_communicator.recv_backward(
    send_tensor_shapes,
    p2p_communicator.is_pp_last_stage,
)
```

Steady State 中常被融合进：

```
send_forward_recv_backward(...)
```

Last Stage 没有下一个 Stage，它的 Backward 从本地 Loss 开始，因此：

```
output_tensor_grad = None
```

第 18 行

```
dx = backward(old, dy)
```

当前 Stage 使用保存的旧 Microbatch 计算图和下游梯度 `dy` 执行反向：

```
dy =  $\partial L / \partial y$ 
当前 Stage:  $y = f(x; \theta)$ 

得到：
```

```
dx = ∂L/∂x
以及本 Stage 参数梯度 ∂L/∂θ
```

真实调用：

```
input_tensor_grad = backward_step(
    input_tensor,
    output_tensor,
    output_tensor_grad,
    config,
)
```

`backward_step()` 的职责：

1. 对 `output_tensor` 运行 PyTorch Autograd；
2. 累积当前 Stage 参数梯度；
3. 读取 `input_tensor.grad`；
4. 返回 `input_tensor_grad`，发送给前一 Stage。

如果启用了 `deallocate_pipeline_outputs`，Megatron 会使用 `custom_backward()` 直接调用 C++ Autograd Engine，因为 `output_tensor.data` 已经被伪释放为标量大小，普通 `torch.autograd.backward` 的 Shape 棠查可能不适用。

第 19 行

```
release_activation(old)
```

教学含义：旧 Microbatch Backward 已完成，它保存的 Activation/Autograd Graph 不再需要。

真实代码没有一个完全同名的单独调用。资源释放来自：

1. `input_tensors.pop(0)` 和 `output_tensors.pop(0)` 移除 FIFO 引用；
2. `backward_step()` 消耗该计算图；
3. 局部 Python 引用离开作用域；
4. PyTorch/CUDA Allocator 把不再引用的显存块回收到缓存池。

必须区分两个时刻：

时刻	释放什么
Forward Output 已发送后	<code>deallocate_output_tensor()</code> 可提前缩小 <code>output_tensor.data</code> ，但保留 <code>grad_fn</code>
Backward 完成后	该 Microbatch 的完整 Autograd 上下文可以回收

第 20 行

```
send_backward(old, dx)
```

把：

```
dx =  $\partial L / \partial x$ 
```

发送给前一个 Pipeline Stage。

真实 API：

```
p2p_communicator.send_backward(  
    input_tensor_grad,  
    p2p_communicator.is_pp_first_stage,  
)
```

First Stage 没有前驱，不需要发送。

Steady State 非最后一轮常组合成：

```
input_tensor = (  
    p2p_communicator.send_backward_recv_forward(  
        input_tensor_grad,  
        recv_tensor_shapes,  
        p2p_communicator.is_pp_first_stage,  
    )  
)
```

同时：

```
向前一个 Stage 发送旧 Microbatch 的 dx  
从前一个 Stage 接收下一个新 Microbatch 的 Forward Input
```

这比教学伪代码中完全串行的 Send/Recv 更接近生产实现。

五、第 21 – 26 行：Cool-down 为什么只剩 Backward

第 22 行

```
for old in range(m - warmup, m):
```

Steady State 已经完成了：

```
m = warmup
```

个旧 Microbatch 的 Backward，仍有 `warmup` 个 Outstanding Microbatch 留在 FIFO 中。

例如：

```
m = 8  
warmup = 3
```

Steady State 完成 B0–B4，剩余：

```
B5、B6、B7
```

第 23 行

```
dy = recv_backward(old)
```

从下一 Stage 接收该旧 Microbatch 的输出梯度。

真实 Cool-down 顺序实际上先从 FIFO 取出对应状态：

```
input_tensor = input_tensors.pop(0)  
output_tensor = output_tensors.pop(0)  
output_tensor_grad = p2p_communicator.recv_backward(...)
```

第 24 行

```
dx = backward(old, dy)
```

使用对应的 `input_tensor`、`output_tensor`、`dy` 执行当前 Stage Backward，得到 `dx` 和本 Stage 参数梯度。

第 25 行

```
release_activation(old)
```

Backward 完成，该 Microbatch 的 Autograd 上下文可以释放。

第 26 行

```
send_backward(old, dx)
```

将输入梯度发送给前一个 Stage。Cool-down 不再接收新的 Forward Input，因为所有  个 Forward 已经完成。

六、真实 Megatron 非交错 1F1B 的核心骨架

下面不是完整源码复制，而是保留真实函数名和 FIFO 行为后的等价骨架：

```
input_tensors = []
output_tensors = []

num_warmup = min(
    p2p_communicator.total_stages
    - p2p_communicator.current_stage
    - 1,
    num_microbatches,
)
num_remaining = num_microbatches - num_warmup

# 1. Warm-up
for microbatch_id in range(num_warmup):
    input_tensor = p2p_communicator.recv_forward(
        recv_tensor_shapes,
        p2p_communicator.is_pp_first_stage,
    )

    output_tensor, num_tokens = forward_step(
        forward_step_func,
        data_iterator,
        model,
        num_microbatches,
        input_tensor,
        forward_data_store,
        config,
        cp_group_size,
        current_microbatch=microbatch_id,
        is_last_stage=p2p_communicator.is_pp_last_stage,
    )

    p2p_communicator.send_forward(
        output_tensor,
        p2p_communicator.is_pp_last_stage,
    )

    input_tensors.append(input_tensor)
    output_tensors.append(output_tensor)

    deallocate_output_tensor(
        output_tensor,
        config.deallocate_pipeline_outputs,
    )

# Steady State 首个 Forward Input
if num_remaining > 0:
    input_tensor = p2p_communicator.recv_forward(
        recv_tensor_shapes,
        p2p_communicator.is_pp_first_stage,
    )

# 2. Steady State
for i in range(num_remaining):
    current_microbatch = i + num_warmup
    last_iteration = i == num_remaining - 1

    output_tensor, num_tokens = forward_step(
```

```

        forward_step_func,
        data_iterator,
        model,
        num_microbatches,
        input_tensor,
        forward_data_store,
        config,
        cp_group_size,
        current_microbatch=current_microbatch,
        is_last_stage=p2p_communicator.is_pp_last_stage,
    )

    output_tensor_grad = (
        p2p_communicator.send_forward_recv_backward(
            output_tensor,
            send_tensor_shapes,
            p2p_communicator.is_pp_last_stage,
        )
    )

    input_tensors.append(input_tensor)
    output_tensors.append(output_tensor)
    deallocate_output_tensor(
        output_tensor,
        config.deallocate_pipeline_outputs,
    )

    old_input = input_tensors.pop(0)
    old_output = output_tensors.pop(0)

    input_tensor_grad = backward_step(
        old_input,
        old_output,
        output_tensor_grad,
        config,
    )

    if last_iteration:
        p2p_communicator.send_backward(
            input_tensor_grad,
            p2p_communicator.is_pp_first_stage,
        )
    else:
        input_tensor = (
            p2p_communicator.send_backward_recv_forward(
                input_tensor_grad,
                recv_tensor_shapes,
                p2p_communicator.is_pp_first_stage,
            )
        )

# 3. Cool-down
for _ in range(num_warmup):
    old_input = input_tensors.pop(0)
    old_output = output_tensors.pop(0)

    output_tensor_grad = p2p_communicator.recv_backward(
        send_tensor_shapes,
        p2p_communicator.is_pp_last_stage,
    )

    input_tensor_grad = backward_step(
        old_input,
        old_output,
        output_tensor_grad,
        config,
    )

    p2p_communicator.send_backward(
        input_tensor_grad,
        p2p_communicator.is_pp_first_stage,
    )

```


真实源码还包含梯度同步开关、Loss 收集、Token 计数、Activation Checkpoint、Multimodule Pipeline、动态 Shape 和计时器等逻辑。

七、最简单的真实 Forward：先不使用多进程

先用两个真实的 `nn.Module` 模拟两个 Pipeline Stage，完整展示 Activation 如何进入下一阶段。

```
import torch
import torch.nn as nn

class Stage0(nn.Module):
    """模拟前半段模型。"""

    def __init__(self, hidden=8):
        super().__init__()
        self.layer = nn.Sequential(
            nn.Linear(hidden, hidden),
            nn.ReLU(),
        )

    def forward(self, x):
        return self.layer(x)

class Stage1(nn.Module):
    """模拟后半段模型。"""

    def __init__(self, hidden=8):
        super().__init__()
        self.layer = nn.Linear(hidden, 1)
        self.input_tensor = None

    def set_input_tensor(self, x):
        # 对应 Pipeline Scheduler 将上游 Activation 注入本 Stage。
        self.input_tensor = x

    def forward(self):
        # 中间/最后 Stage 不需要重新从 Tokens 开始，
        # 而是消费上一个 Stage 产生的 Hidden States。
        return self.layer(self.input_tensor)

torch.manual_seed(0)

stage0 = Stage0(hidden=8)
stage1 = Stage1(hidden=8)

# 一个最小 Microbatch: [micro_batch=2, hidden=8]
x = torch.randn(2, 8, requires_grad=True)
target = torch.randn(2, 1)

# Stage 0 Forward
h = stage0(x)
print("stage0 output:", h.shape)      # [2, 8]

# 模拟跨 Pipeline Rank 传输边界：
# 真正分布式通信后，接收端拿到的是新的叶子 Tensor。
h_recv = h.detach().requires_grad_(True)

# Scheduler 将接收到的 Activation 注入 Stage 1。
stage1.set_input_tensor(h_recv)
```

```
# Stage 1 Forward
prediction = stage1()
print("stage1 output:", prediction.shape) # [2, 1]

loss = ((prediction - target) ** 2).mean()
print("loss:", float(loss))

# Last Stage 从 Loss 开始 Backward。
loss.backward()

# h_recv.grad 就是要发送回 Stage 0 的 dy = dLoss/dh。
dh = h_recv.grad
print("activation grad:", dh.shape) # [2, 8]

# Stage 0 使用下游返回的 dh 继续反向。
h.backward(dh)
print("x grad:", x.grad.shape) # [2, 8]
```

这段代码的关键不是模型本身，而是这两行：

```
h_recv = h.detach().requires_grad_(True)
stage1.set_input_tensor(h_recv)
```

为什么要 `detach()` ？

真实跨进程 Send/Recv 不会把 Stage 0 的 Python Autograd Graph 自动传到 Stage 1。接收端的 Activation 是一个新的 Tensor，需要：

```
requires_grad_(True)
```

这样 Stage 1 Backward 后才能得到：

```
h_recv.grad = ∂Loss/∂h
```

然后把这个梯度发送回 Stage 0，Stage 0 执行：

```
h.backward(dh)
```

这正对应伪代码：

```
Forward: send y
Backward: recv dy → backward → send dx
```

八、最简单的两进程 Forward 通信示例

下面只演示 Forward 的真实跨进程通信，不实现完整 1F1B 调度。运行要求：两张 GPU，使用 `torchrun --nproc_per_node=2`。

```

# minimal_pipeline_forward.py
import os
import torch
import torch.distributed as dist
import torch.nn as nn

def main():
    dist.init_process_group(backend="nccl")

    rank = dist.get_rank()
    torch.cuda.set_device(rank)
    device = torch.device(f"cuda:{rank}")

    micro_batch = 2
    hidden = 8

    if rank == 0:
        # Pipeline Stage 0
        stage0 = nn.Sequential(
            nn.Linear(hidden, hidden),
            nn.ReLU(),
        ).to(device)

        x = torch.randn(
            micro_batch,
            hidden,
            device=device,
        )

        # 当前 Stage 真实 Forward。
        activation = stage0(x)
        print("rank0 activation:", activation.shape)

        # Forward P2P: 发送给下一个 Pipeline Rank。
        # send 只传 Tensor 数据, 不传 Python Autograd Graph。
        dist.send(activation.detach(), dst=1)

    elif rank == 1:
        # Pipeline Stage 1
        stage1 = nn.Linear(hidden, 1).to(device)

        # 接收端必须提前分配正确 Shape/Dtype 的 Tensor。
        activation = torch.empty(
            micro_batch,
            hidden,
            device=device,
        )

        # Forward P2P: 从前一个 Pipeline Rank 接收。
        dist.recv(activation, src=0)
        activation.requires_grad_(True)

        output = stage1(activation)
        print("rank1 output:", output.shape)

    dist.barrier()
    dist.destroy_process_group()

if __name__ == "__main__":
    main()

```

运行:

```
torchrun --standalone --nproc_per_node=2 minimal_pipeline_forward.py
```

这个例子对应：

```
Rank 0: forward → send_forward  
Rank 1: recv_forward → forward
```

生产 Megatron 不直接在业务代码里写裸 `dist.send/dist.recv`，而是通过：

```
P2PCommunicator.recv_forward(...)  
P2PCommunicator.send_forward(...)
```

原因包括：

- 管理 Pipeline Process Group；
- First/Last Stage 特殊处理；
- Tensor Shape 协议；
- Batched P2P；
- NCCL Request 等待；
- 与反向收发组合；
- 通信重叠配置；
- 多 Tensor 输入输出。

九、Megatron 风格的用户 `forward_step_func`

Megatron 把“调度”与“模型业务 Forward”分开。

调度器大致这样调用：

```
forward_backward_func = get_forward_backward_func()  
  
losses = forward_backward_func(  
    forward_step_func=forward_step_func,  
    data_iterator=data_iterator,  
    model=model,  
    num_microbatches=num_microbatches,  
    seq_length=seq_length,  
    micro_batch_size=micro_batch_size,  
    forward_only=False,  
)
```

用户提供的 `forward_step_func` 通常返回：

```
模型输出 + Loss Reduction Callback
```

最小教学版：

```
from functools import partial
import torch

def loss_func(target, output_tensor):
    loss = torch.nn.functional.mse_loss(
        output_tensor,
        target,
    )

    # 第二项是记录/日志元数据。
    return loss, {"mse": loss.detach()}

def forward_step_func(data_iterator, model):
    batch = next(data_iterator)

    # First Stage 需要真正的模型输入。
    # Middle Stage 的真正输入已由 scheduler 通过
    # model.set_input_tensor(input_tensor) 注入。
    model_input = batch.get("input")
    target = batch.get("target")

    output_tensor = model(model_input)

    # 绑定当前 Microbatch 的 target。
    return output_tensor, partial(loss_func, target)
```

真实 GPT 示例更接近：

```
def forward_step(data_iterator, model):
    (
        tokens,
        labels,
        loss_mask,
        attention_mask,
        position_ids,
    ) = get_batch(data_iterator)

    output_tensor = model(
        tokens,
        position_ids,
        attention_mask,
        labels=labels,
        loss_mask=loss_mask,
    )

    return output_tensor, partial(
        loss_func,
        loss_mask,
        model=model,
    )
```

不过 Pipeline 情况下要理解两类输入：

输入类型	来源	示例
Dataset-Side Input	<code>data_iterator/get_batch</code>	Tokens、Labels、Loss Mask、Position IDs
Pipeline Activation Input	前一个 PP Rank，经 P2P 接收并 <code>set_input_tensor</code>	Hidden States

First/Middle/Last Stage 的差异：

Stage	Dataset Input	Pipeline Activation	输出
First	Tokens 等	无前驱 Activation	Hidden States, 发送给下一 Stage
Middle	通常只需少量元数据或全 <code>None</code>	从上一 Stage 接收	新 Hidden States
Last	Labels、Loss Mask 等	从上一 Stage 接收	Logits/Per-Token Loss, 计算最终 Loss

因此，中间 Stage 即使：

```
tokens = None
```

仍能运行，因为真正网络输入来自：

```
recv_forward
→ model.set_input_tensor(received_hidden_states)
→ model.forward(...)
```

十、伪代码中还隐藏了哪些生产细节

1. 通信 Shape

教学代码只写：

```
recv_forward(i)
```

真实代码必须知道接收 Tensor 的 Shape/Dtype。固定序列长度时可预先计算；变长序列时可能先交换 Shape。

2. First/Last Stage 分支

- First Stage：不 `recv_forward`；
- Last Stage：不 `send_forward`，也不 `recv_backward`；
- Last Stage Backward 从 Loss 开始；
- First Stage 不 `send_backward`。

3. 组合通信

Steady State 常用：

```
send_forward_recv_backward(...)
send_backward_recv_forward(...)
```

而不是四个完全分离、串行的调用。

4. 梯度同步

Megatron 通常在 Microbatch Accumulation 期间暂时关闭 DDP 梯度同步，仅在本 Stage 最后一个 Backward 恢复同步，避免每个 Microbatch 都触发一次全局梯度通信。

5. Activation Checkpoint

不同 Microbatch 可以选择 Partial/Full Activation Checkpoint，改变 FIFO 中计算图保存量与重算代价。

6. VPP Model Chunk

VPP 模式下 `model` 不再只是单个 `nn.Module`，而可能是多个 Model Chunks 的列表；调度器还要维护 Virtual Pipeline Rank，选择当前时刻执行哪个 Chunk。

7. Output Pseudo-Deallocation

输出发送后可执行：

```
deallocate_output_tensor(...)
```

减少输出数据存储，但保留 `grad_fn`。稍后反向可能使用 `custom_backward()` 绕过普通 Shape 检查。

8. Loss 只在 Last Stage 真正产生

非 Last Stage 的 `output_tensor` 是传给后续 Stage 的 Activation；Last Stage 才将模型输出交给 Loss Callback。

十一、完整数据生命周期

以某个 Microbatch `MB0` 为例：

```
1. Stage 0 从 Dataset 读取 Tokens
2. Stage 0 Forward 得到 h0
3. h0 发送到 Stage 1
4. Stage 0 把 input0/output0 引用放入 FIFO
5. Stage 1 接收 h0, 设为 requires_grad 的 Pipeline Input
6. Stage 1 Forward 得到 h1
7. 继续传到最后 Stage
8. Last Stage 计算 Loss
9. Last Stage Backward 得到 dL/dh_prev
10. 梯度逐 Stage 向前发送
11. Stage 0 从 FIFO 取出 MB0 的 input/output
12. Stage 0 使用收到的 dy 执行 backward_step
```

- 13. 累积本 Stage 参数梯度
- 14. MB0 的 Autograd 上下文可回收

1F1B 的关键就是：当 MB0 在向后 Backward 时，较新的 MB3/MB4 等可能正在 Forward。

十二、最终记忆方式

把伪代码压缩成四个状态变换：

```
Forward Input
  ↓ recv_forward / 本地 Dataset
Current Stage Forward
  ↓
Forward Output
  ├── send_forward 给下游
  └── 保存 input/output/grad_fn 等待反向

下游返回 output gradient dy
  ↓
backward_step(old_input, old_output, dy)
  ├── 累积本 Stage 参数梯度
  └── 得到 input gradient dx
      ↓
      send_backward 给上游
```

对应 1F1B：

```
Warm-up: 只增加 Outstanding FIFO
Steady: 每轮入队一个新 Forward、出队一个旧 Backward
Cool-down: 只清空 Outstanding FIFO
```

这就是截图中全部代码真正表达的机制。

可选自测

1. 在 Stage 1 的 Steady State 中，当前正在做 **F4**，同时做 **B1**。
`send_forward_recv_backward()` 发出的和收到的分别属于哪个 Microbatch?
2. 为什么跨 Rank 收到的 Activation 需要成为 `requires_grad=True` 的新 Tensor？为什么不能把 Stage 0 的 Autograd Graph 自动跨进程传过去？
3. `deallocate_output_tensor()` 为什么可以缩小 `output_tensor.data`，却仍然能在之后执行 Backward？它保留了什么？

事实来源与版本边界

本文对真实调用的描述依据截至 2026-07-29 的 NVIDIA Megatron Core 官方 API 文档，以及 NVIDIA/Megatron-LM `main` 分支中的 `megatron/core/pipeline_parallel/schedules.py`、`p2p_communication.py` 和 `pretrain_gpt.py`。具体参数、Multimodule 支持、组合通信与 VPP 调度细节会随版本演进；生产排查应以实际安装版本源码为准。

十七、最常见的五个误区

误区 1: 1F1B 把普通 PP 的 Bubble 消掉了

没有。普通 1F1B 主要降低 Activation 存储，经典理想 Bubble 与 GPipe 相同。

误区 2: $(p-1)/m$ 是 Bubble 占总时间百分比

不严格。它是 Bubble 相对 Ideal/Useful 时间的比值。真正占实际总时间的是：

$$(p-1)/(m+p-1)$$

误区 3: VPP 有 pv 个逻辑 Stage，所以 Useful 应乘 v

不对。每个 Chunk 只有原 Stage 约 $1/v$ 的计算量， v 个 Chunk 加起来仍是原来的 Stage 工作量。

误区 4: Bubble 减半，所以训练速度翻倍

不对。只减少空闲部分，有效计算不变。本例理想速度只提升约 15.8%。

误区 5: VPP 的 v 越大越好

不对。Bubble 约按 $1/v$ 下降，但通信量约按 v 增加，还会增加调度复杂度、Kernel Launch 和 Buffer 压力。

十八、最终心智模型

普通 PP:
每张 GPU 一个连续大 Stage
→ Microbatch 形成流水线
→ 开头需要 Fill, 结尾需要 Drain
→ $Bubble=(p-1)(t_f+t_b)$

普通 1F1B:
Warm-up 后交替 Forward/Backward
→ 更早释放 Activation
→ 显存从约 $O(m)$ 降至 $O(p)$

→ 但 Fill/Drain 依赖仍在, Bubble 不变

VPP:

每张 GPU 的大 Stage 切成 v 个小 Chunks

→ 同一 GPU 扮演多个虚拟 Stage

→ 不可避免的空档粒度从 Stage 时间缩短到 Chunk 时间

→ Bubble 理想缩小约 v 倍

→ Useful 不变, P2P 通信约增加 v 倍

判断 VPP 是否值得使用, 只看一个净收益式:

VPP 净收益

≈ 节省的 Bubble

- 新增的暴露 P2P 通信
- 新增调度/Launch 开销
- 负载不均放大项

可选自测

1. 当 $p=8$ 、 $m=32$ 、 $t_f=2\text{ ms}$ 、 $t_b=3\text{ ms}$ 时, 普通 1F1B 的 T_{useful} 、 T_{bubble} 、 T_{total} 、Bubble/Actual 分别是多少?
2. 在上题基础上使用 $v=4$, 理想 VPP 的 Bubble、Total 和理想加速比是多少?
3. 为什么普通 1F1B 能显著减少 Activation Memory, 却不能像 VPP 那样把理论 Bubble 缩小? 请从依赖链和任务粒度解释。

通信代价与 overlap

普通 PP 的每个物理 stage 边界, 每个 microbatch 前向发送 activation、反向发送 activation gradient。VPP 把总逻辑 stage 数从 p 增加到 $p \times v$, 边界也增加, 因此 NVIDIA 给出的结论是总 P2P 通信量约增加 v 倍。消息 tensor shape 通常仍是 $[\text{micro_batch}, \text{sequence}, \text{hidden}]$, 不是每条消息缩小 v 倍; 增加的是边界/消息次数。

可以 overlap: 调度器可用异步 P2P send/recv, 在某个 chunk 通信时执行本 GPU 另一个 chunk 或另一个 microbatch 的计算。但是否真重叠取决于 NCCL/P2P backend、CUDA stream、网络拓扑和 SM/显存带宽争用。若通信暴露时间超过节省的 bubble, 继续增大 v 反而变慢。必须用 Nsight Systems/Megatron timers 看 timeline。

显存影响

VPP 不会像 FSDP 那样把参数、梯度、优化器状态降到 $1/N$ ；每张 GPU 仍持有与普通 PP 大致相同总层数，只是拆成多个不连续 chunks。参数显存基本不变。activation 显存由调度中同时在飞的 microbatches、chunk 数、recompute 策略决定：VPP 不是天然的显存优化，某些配置下 bookkeeping/通信 buffer/保存 activation 反而更复杂。1F1B 的核心内存优势是把 outstanding forward activations 从 $O(m)$ 限制到与 pipeline 深度同阶；实际 VPP 峰值应以框架实现和 profiler 为准。

Megatron 配置示例

```
# Megatron Core / NeMo Bridge 风格（当前官方文档）
model_config = GPTModelProvider(
    num_layers=16,
    pipeline_model_parallel_size=4,
    virtual_pipeline_model_parallel_size=2, # 每个物理 rank 2 个 chunks
)

# Megatron-LM 某些 CLI 版本使用“每个虚拟 stage 的层数”表达
--pipeline-model-parallel-size 4 \
--num-layers-per-virtual-pipeline-stage 2

# 本例:  $v = L / (p \times \text{layers\_per\_virtual\_stage})$ 
#       =  $16 / (4 \times 2) = 2$ 

# 版本敏感：不同 Megatron/NeMo 版本参数名与约束可能不同，
# 以当前安装版本的官方 config/API 为准。
```

实现约束与调优方法

常见约束：层数要能被物理 stage 数与 virtual chunk 粒度整除；Megatron 某些版本要求 microbatch 数能被 pipeline parallel size 整除；部分旧实现对 $p=2$ 的 interleaved schedule 有限制。调优建议：1) 先保证物理 stage 计算均衡；2) 固定 p, m ，从 $v=2$ 开始；3) 比较 bubble time、P2P exposed time、activation peak、step time；4) 只有 bubble 节省大于额外通信时才提高 v ；5) TP 放机内 NVLink，PP/VPP 边界尽量按拓扑映射，避免高频跨慢链路往返。

与相邻技术的区别

普通 PP：每 GPU 一个连续 stage；VPP：每 GPU 多个不连续 chunks，interleaved 调度，bubble 更小但通信更多。

1F1B：是一类调度策略，普通 PP 与 VPP 都可用；VPP 常指 interleaved 1F1B。

Zero-Bubble Pipeline: 通常进一步拆分 backward 的 input-gradient(B) 与 weight-gradient(W) 并重排, 以逼近零 bubble; VPP 主要靠把 stage 切成 chunks, 并不等于 zero-bubble。
Tensor Parallel: 切单个算子张量; VPP 仍是按层深度切分。
FSDP/DP: 切数据和训练状态; 与 VPP 正交, 可组成 TP×PP/VPP×DP/FSDP。

什么时候应该用

适合: 已经必须使用 PP; 普通 1F1B 的 profiler 显示 bubble 明显; microbatch 数受 global batch 或显存限制不能继续增大; P2P 链路足够快; 模型层数足够多且能均匀切 chunk。

不适合: p 很小或 $m \gg p$, 原 bubble 已很低; 网络慢、P2P 已是瓶颈; 层数不能均匀切; stage 本身严重不均衡; 显存极紧且 VPP 调度增加 activation/buffer 峰值。此时先解决负载均衡、拓扑映射、recompute 或减少通信。

常见误区

- 1) VPP 不是创建更多 GPU, 也不是让一张 GPU 真正同时跑两个大 GEMM; 它创建多个逻辑 stage, 并通过时间调度提高利用率。
- 2) v 越大不一定越快: bubble 下降约 $1/v$, 但通信量约增 v , 存在最优点。
- 3) VPP 不等于显存分片: 每卡总参数基本不变。
- 4) 不要混淆两种 bubble 百分比口径。
- 5) 理论公式忽略通信和不均衡; 最终决策必须看真实 step time 和 timeline。

关键总结

VPP = PP 的更细粒度 stage 切分 + interleaved 1F1B。它让每张 GPU 持有 v 个 model chunks, 用更短的 chunk 计算填补流水线等待; 理想 bubble 缩小 v 倍, 但 P2P 通信约增加 v 倍。选择 v 的本质是用更多通信换更少空闲时间, 工程上必须测量 bubble 节省是否超过通信与调度开销。

可选自测

1. 因果题: 为什么 VPP 的 bubble 理想上缩小约 v 倍, 但总通信量反而约增加 v 倍?
2. 计算题: $p=8$ 、 $m=16$ 、 $v=4$ 时, 分别计算普通 1F1B 与 VPP 的 bubble/useful 和 bubble/total。

3. 工程题：若 Nsight 显示普通 PP bubble 只有 5%，但 P2P 已占 step time 的 18%，是否应该启用 $v=2$ ？说明理由。

继续学习

1. 前置：Pipeline Parallel 与 1F1B 的完整时间线
2. 同层对比：Zero-Bubble Pipeline Parallelism
3. 下游应用：TP $\times$ VPP $\times$ DP/FSDP 的 3D 并行拓扑布局

事实来源与版本边界

核心公式和“通信量随 v 增加”的结论来自 NVIDIA Megatron 官方技术文章；配置字段参考 Megatron Core 0.16 与 NeMo Megatron Bridge 文档。截至 2026-07-27，不同 Megatron/NeMo 版本的 CLI 参数名和约束可能变化，生产配置应核对实际安装版本。